Taller 2

Maria Fernanda Gordillo Robles - 122117

Karen Yulieth Montaña Sanchez - 123757

Adrián David Sierra Chavarro - 122228

Facultad de Ingeniería Industrial

Universidad ECCI

Sistemas Avanzados de Producción

2026

Introducción

El presente taller aborda la implementación y análisis de modelos de aprendizaje automático orientados a la solución de problemas complejos en Ingeniería Industrial, mediante el uso de técnicas avanzadas de regresión regularizada, optimización numérica y redes neuronales artificiales. El enfoque se centra en la modelación de sistemas no lineales, donde las relaciones entre variables presentan alta dimensionalidad, multicolinealidad y comportamientos estocásticos.

Se estudian dos casos aplicados: la predicción de la demanda en sistemas de micro movilidad urbana y la clasificación de fallos en maquinaria industrial bajo esquemas de mantenimiento predictivo. Estos problemas implican retos asociados a la generalización del modelo, el desbalance de clases y la correcta selección de métricas de evaluación, aspectos críticos para garantizar la validez de los resultados en contextos reales.

El desarrollo metodológico integra la derivación formal de la regularización Ridge, la implementación de algoritmos de optimización basados en descenso de gradiente y el diseño de arquitecturas de redes neuronales multicapa (MLP), incluyendo el ajuste de hiper parámetros y el análisis del proceso de propagación hacia adelante. Asimismo, se enfatiza la evaluación del desempeño mediante métricas robustas y su interpretación en función del impacto operativo.

En este sentido, el taller permite consolidar competencias en la formulación, entrenamiento y validación de modelos predictivos, orientados a la toma de decisiones basada en datos dentro de sistemas productivos y logísticos complejos.

Fase 1: Regularización Ridge — Derivación Matemática

La regresión Ridge constituye una extensión de los Mínimos Cuadrados Ordinarios (OLS) que incorpora un término de regularización tipo L2. Este término penaliza la magnitud de los coeficientes del modelo, contribuyendo a mejorar su capacidad de generalización, especialmente en escenarios donde existe **multicolinealidad** o alta dimensionalidad en los datos.

El objetivo principal es estabilizar la estimación de los parámetros, reduciendo la varianza del modelo sin incrementar excesivamente el sesgo.

1.1 Representación vectorial de la función de costo

La función de costo de Ridge puede expresarse inicialmente en forma escalar y posteriormente en notación matricial compacta.

Forma escalar

$$(\beta) = \Sigma_i(y_i - \beta_0 - \Sigma\ \beta\ x_i\ )^2 + \lambda\ \Sigma\ \beta^{\ 2}$$

Forma matricial

$$J(\beta) = (Y - X\beta)^T(Y - X\beta) + \lambda\beta^T\beta$$

Donde:

$Y \in \mathbb{R}^n \rightarrow$ vector columna de respuestas observadas

$X \in \mathbb{R}^{nxp} \rightarrow$ matriz de diseño (incluye columna de 1s para β_0)

$\beta \in \mathbb{R}^p \rightarrow$ vector de parámetros a estimar $\lambda \geq 0 \rightarrow$ hiperparámetro de regularización

La suma $\Sigma\beta^2$ equivale al producto interno $\beta^T\beta = \|\beta\|^2$. Esta representación permite expresar la penalización como un término matricial algebraico y diferenciable, conservando la forma cuadrática de $J(\beta)$ y asegurando la existencia de un único mínimo global.

1.2 Expansión de la forma cuadrática

Se procede a expandir el término principal utilizando propiedades de álgebra matricial:

$$(Y - X\beta)^T(Y - X\beta)$$

Desarrollando:

$$(Y - X\beta)^T(Y - X\beta): = Y^TY - Y^T(X\beta) - (X\beta)^TY + (X\beta)^T(X\beta)$$

Usando $(X\beta)^T = \beta^TX^T$ y $Y^T(X\beta) = (X\beta)^TY$

$$\text{(ambos son escalares)}: = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$$

Sumando el término de penalización $\lambda\beta^T\beta$:

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$$

Factorizando los términos cuadráticos en β:

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

Donde **I** es la matriz identidad de dimensión (p \times p) (igual a la de (X^TX)). Esta matriz inversa existe para todo (\lambda > 0), independientemente de que (X^TX) sea singular.

Para finalizar esta primera fase, daremos las respuesta a las preguntas conceptuales:

1. ¿Por qué la suma λI garantiza que la matriz resultante sea invertible incluso si X^TX es singular?

La matriz X^TX es semidefinida positiva, por lo que puede presentar valores propios nulos en presencia de multicolinealidad o cuando (p > n), lo que implica pérdida de rango e imposibilidad de inversión. La adición del término de regularización lambda, con lambda > 0), desplaza el espectro de valores propios, garantizando que la matriz resultante sea definida positiva e invertible. Desde una perspectiva geométrica, este ajuste elimina las direcciones degeneradas del espacio de parámetros, asegurando una solución única y estable. En contextos como el dataset *Bike Sharing*, caracterizado por alta correlación entre variables, esta propiedad es esencial para obtener estimaciones robustas.

2. ¿Qué sucede con el modelo cuando $\lambda \to 0$? ¿Y cuando $\lambda \to \infty$?

Cuando es $\lambda \to 0$ La penalización desaparece, por lo que el estimador se reduce al de Mínimos Cuadrados Ordinarios. En consecuencia, los coeficientes pueden tomar valores elevados, otorgando al modelo máxima flexibilidad, pero también una alta varianza. Esto incrementa el riesgo de sobreajuste, ya que el modelo tiende a capturar no solo la señal, sino también el ruido presente en los datos de entrenamiento, mientras $\lambda \to \infty$ La penalización domina la función de costo, forzando a los coeficientes a aproximarse a cero. Como resultado, el modelo se vuelve excesivamente simple y pierde capacidad de ajuste frente a los datos. Aunque la varianza se reduce significativamente, se introduce un sesgo elevado que limita la capacidad predictiva del modelo.

Explicado en términos de Sesgo -Varianza

Trade-off Sesgo-Varianza:

Error Total = Sesgo² + Varianza + Ruido irreducible

λ pequeño → sesgo bajo, varianza alta (sobreajuste)

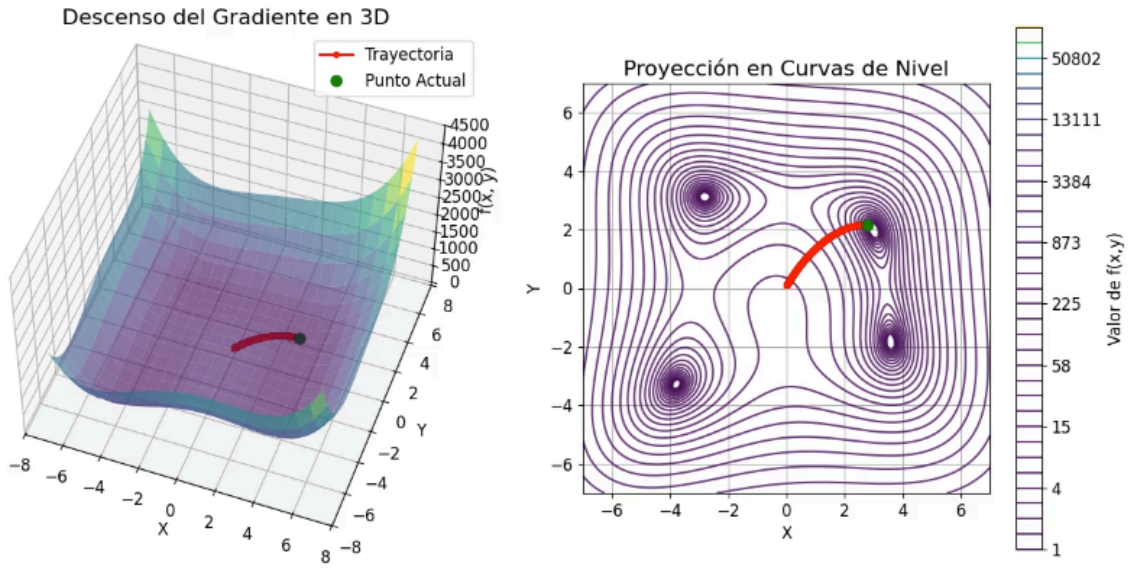
λ óptimo → balance entre sesgo y varianza

λ grande → sesgo alto, varianza baja (subajuste)

Fase 2.

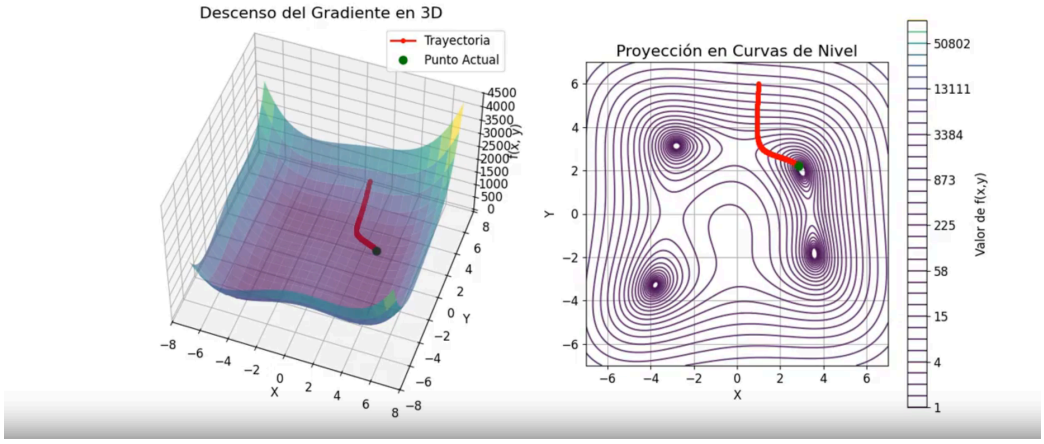
Se utilizó la función de Himmelblau $f(x,y) = (x^2+y-11)^2 + (x+y^2-7)^2$, la cual presenta cuatro mínimos locales, como superficie de prueba para el algoritmo de descenso de gradiente. Se llevaron a cabo cuatro experimentos, variando la tasa de aprendizaje η\etaη y el punto inicial.

1. Figura [1]: Experimento C: η no óptimo — convergencia parcial



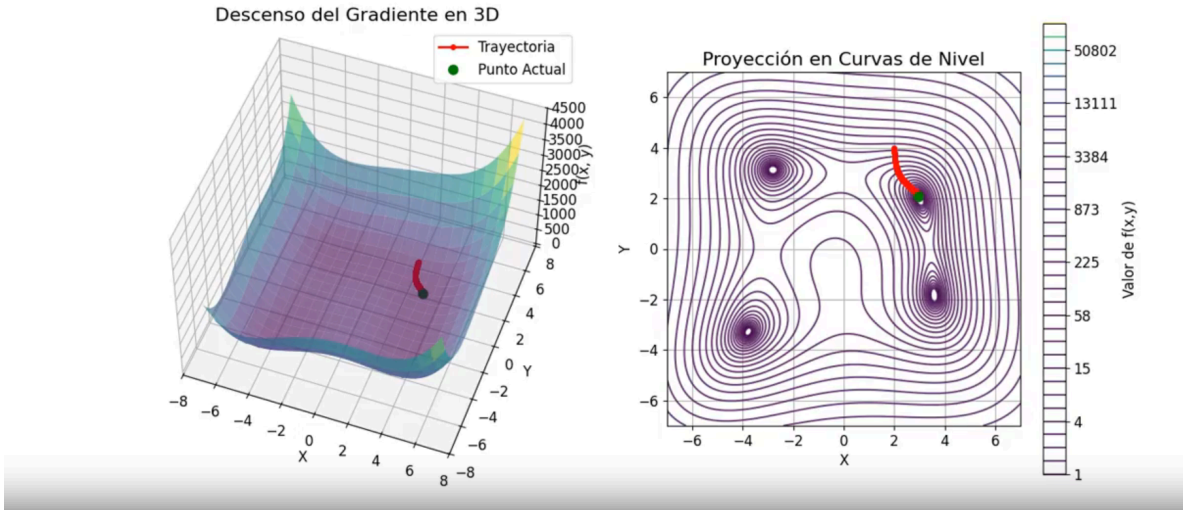
La trayectoria parte desde el origen y avanza hacia una región de menor valor, pero se detiene antes de alcanzar el mínimo global. El valor de esta no es adecuado, lo que limita la convergencia, quedando atrapado en una zona intermedia.

Figura 2. Punto inicial [1, 6]



Al iniciar el descenso desde [1,6], la trayectoria muestra cómo el algoritmo baja desde una región elevada de la superficie, pero se detiene en una zona intermedia sin alcanzar un mínimo profundo. Esto refleja una **convergencia parcial**, donde la tasa de aprendizaje no logra guiar el punto hacia una solución óptima. El comportamiento evidencia que, aunque se reduce el error inicial, el algoritmo queda atrapado en valores relativamente altos, lo que confirma la sensibilidad del método al punto de inicio y a la calibración del parámetro de aprendizaje.

Figura 3. Punto inicial [2, 4]



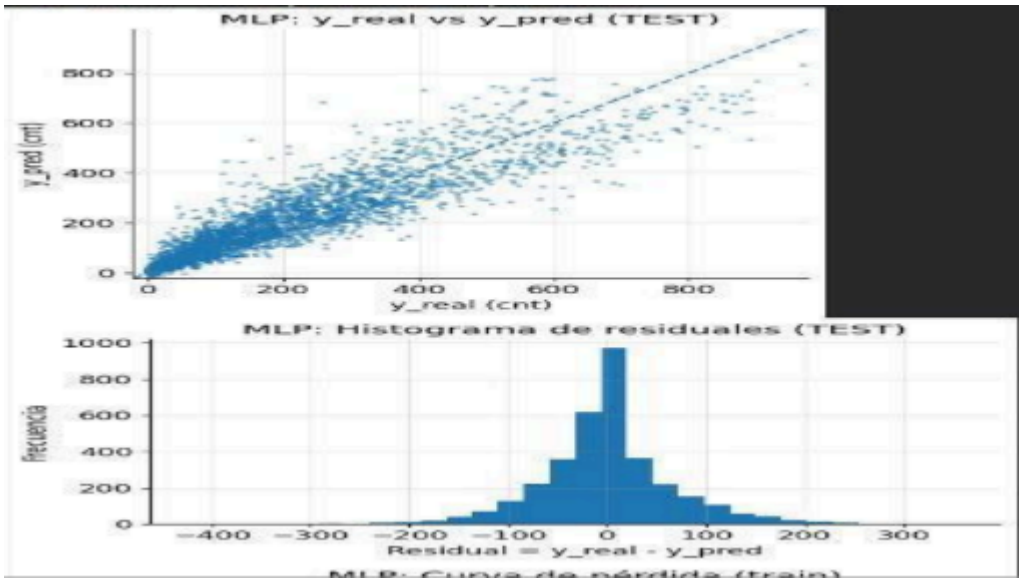
Desde el punto [2,4], la trayectoria desciende de manera más ordenada y estable, curvándose suavemente hacia un mínimo local. En la proyección de curvas de nivel se aprecia cómo el camino se dirige con pasos consistentes hacia una región de menor valor de f(x,y), sin oscilaciones bruscas ni estancamientos. Esto confirma que la tasa de aprendizaje utilizada es adecuada para esta región de la superficie y que el algoritmo logra una convergencia clara y consistente. Sin embargo, el mínimo alcanzado depende del punto

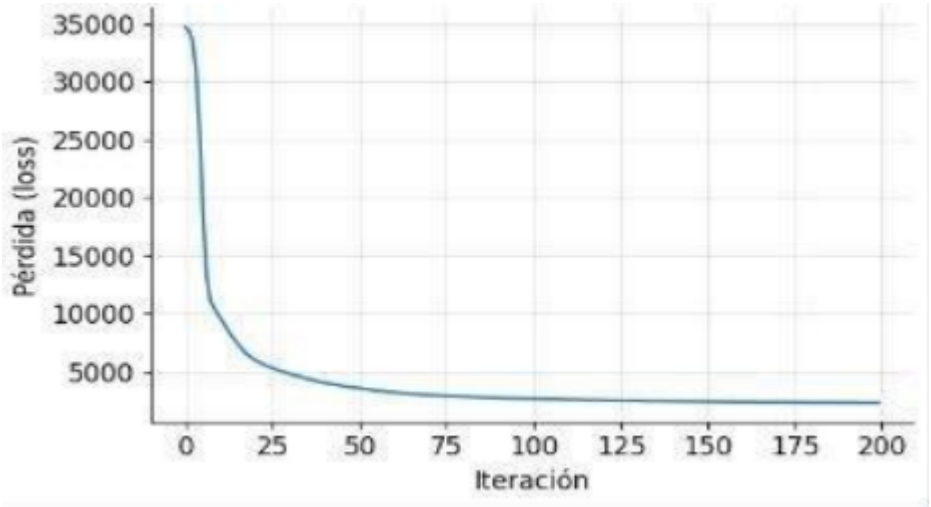
inicial, lo que demuestra la presencia de múltiples mínimos locales en la función de Himmelblau y la necesidad de estrategias adicionales para evitar soluciones subóptimas.

- Fenómeno de Overshooting: Cuando el paso de actualización es mayor que la distancia real al mínimo, el algoritmo rebota fuera del valle.
    - El producto $\eta \cdot \nabla f$ genera un salto demasiado grande.
    - En lugar de acercarse al mínimo, el punto cruza al lado opuesto.
    - En el gráfico de curvas de nivel se visualiza como una trayectoria zigzagueante que nunca converge, rebotando entre las paredes del valle.
- Eficiencia Computacional: La estrategia correcta es ajustar la tasa de aprendizaje, no solo aumentar iteraciones.
    - Más iteraciones con un η inadecuado solo repiten el mismo error.
    - Ajustar η permite pasos proporcionales a la geometría de la superficie.
    - Métodos como learning rate scheduling o algoritmos adaptativos (Adam) mejoran la convergencia.
- Generalización en problemas complejos: Con cientos de variables, un único punto inicial puede llevar a mínimos locales subóptimos.
    - Funciones de alta dimensión tienen miles de mínimos locales
    - Confiar en un solo inicio puede atrapar el algoritmo en soluciones pobres
    - Estrategias como **reinicios aleatorios**, momentum o búsqueda global ayudan a escapar de mínimos locales
-

Fase 3.

Fase 3A.





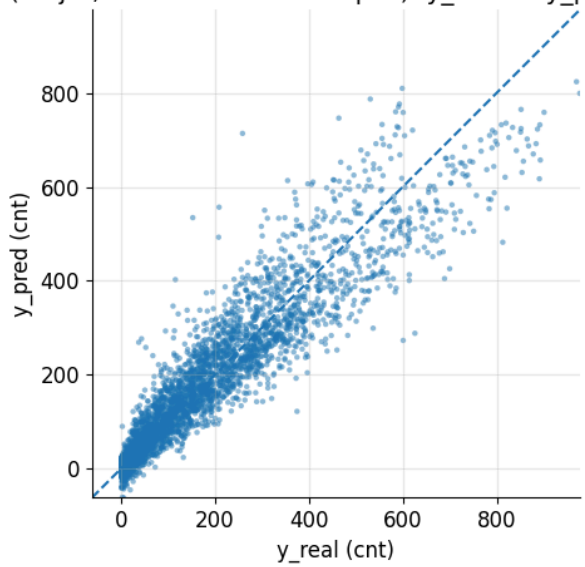
- El MAE refleja el error promedio en términos absolutos. En este caso, el modelo equivoca en promedio unas ~46 bicicletas por hora. Esta cifra es fácil de interpretar: si un operador planifica la redistribución de bicicletas, puede esperar que el modelo se desvíe alrededor de esa cantidad en condiciones normales. Es una métrica útil para comunicar resultados a un público no técnico porque traduce directamente el error en unidades físicas.

- El RMSE amplifica los errores grandes, ya que eleva al cuadrado las diferencias antes de promediarlas. Aquí se observa un valor cercano a ~69 bicicletas. Esto indica que, aunque la mayoría de las predicciones son razonables, existen ciertos momentos (como horas pico) donde el error es mucho mayor y afecta de forma desproporcionada la métrica. En términos operativos, el RMSE alerta sobre los riesgos de subestimar la demanda en escenarios críticos.
- El R^2 cercano a 0.85 muestra que el modelo explica aproximadamente el 85% de la variabilidad en la demanda. Esto significa que las variables de entrada (hora, clima, estacionalidad) capturan gran parte del comportamiento del sistema, aunque todavía queda un 15% de variabilidad no explicada, posiblemente asociada a factores externos como eventos especiales o fluctuaciones sociales.

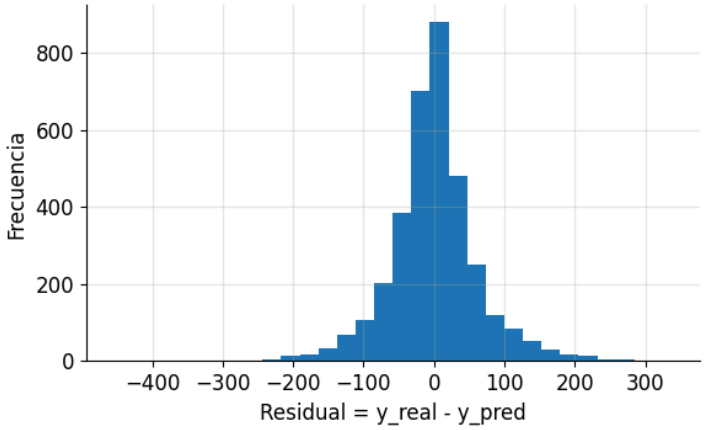
Fase 3B.

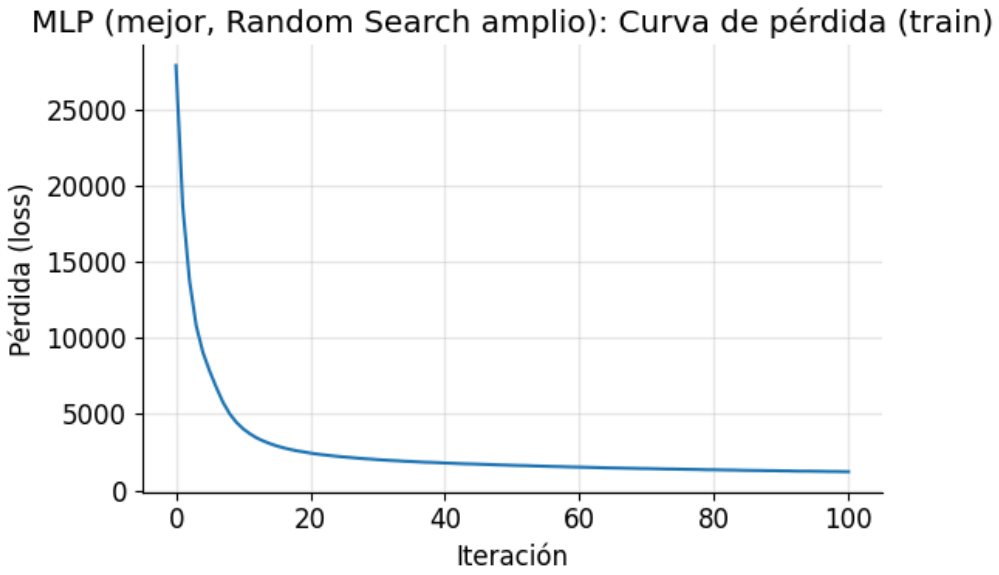
```
Ejecutando RandomizedSearchCV con espacio AMPLIO...
Fitting 3 folds for each of 50 candidates, totalling 150 fits
Fitting 3 folds for each of 50 candidates, totalling 150 fits
RandomizedSearchCV tomó: 3533.33 s

== Resultado de la búsqueda aleatoria (CV) ==
Mejores hiperparámetros: {'learning_rate_init': 0.01, 'learning_rate': 'adaptive', 'hidden_layer_sizes': (128,), 'alpha': 0.01, 'activation': 'tanh'}
RMSE CV (aprox): 68.520692943139

== MLP (MEJOR MODELO — Random Search amplio) ==
Train -> MAE: 36.438 | RMSE: 53.977 | R2: 0.912
Test  -> MAE: 44.566 | RMSE: 64.516 | R2: 0.869
```

MLP (mejor, Random Search amplio): y_real vs y_pred (TEST)



MLP (mejor, Random Search amplio): Histograma de residuales (TEST)





- El valor óptimo de alpha encontrado mediante el Random Search fue 0.01, lo cual indica un nivel de regularización intermedio dentro del rango evaluado [1e−5,1e−2,1.0] [1e−5,1e−2,1.0]. Este resultado es consistente con lo observado en la Regresión Ridge, donde el parámetro alpha también controla la regularización L2. En ambos modelos, un alpha muy bajo puede generar sobreajuste, mientras que un alpha muy alto puede provocar subajuste. El valor seleccionado (0.01) sugiere un equilibrio adecuado entre ambos extremos. Además, el mejor modelo corresponde a una red con arquitectura (128,), es decir, una red relativamente amplia. Este nivel de complejidad implica una mayor cantidad de parámetros, lo que incrementa el riesgo de sobreajuste. Por esta razón, es coherente que el modelo haya seleccionado un valor de alpha moderado para controlar dicha complejidad. Por lo tanto, se confirma que existe una relación directa entre la complejidad de la red y la necesidad de regularización: modelos más complejos requieren mayor regularización para generalizar correctamente, tal como ocurre también en la Regresión Ridge.

- El uso de Early Stopping fue clave para mejorar la eficiencia y el desempeño del modelo. En este caso, se configuró con early_stopping=True, una fracción de validación del 20% y un criterio de parada de 15 iteraciones sin mejora. Este mecanismo permitió reducir el tiempo de entrenamiento, ya que el modelo no necesitó completar las 1200 iteraciones máximas si dejaba de mejorar antes. Esto es especialmente importante considerando que el proceso de búsqueda tomó aproximadamente 3533 segundos, por lo que optimizar cada entrenamiento individual resulta fundamental. Por otro lado, Early Stopping ayudó a prevenir el sobreajuste, deteniendo el entrenamiento en el punto donde el modelo alcanzó su mejor desempeño en validación. Esto se refleja en los resultados obtenidos:

  Entrenamiento: $R^2 = 0.912$

  Prueba: $R^2 = 0.869$

  La diferencia entre ambos valores es relativamente pequeña, lo que indica que el modelo generaliza bien y no está sobre ajustando significativamente.

- Conclusión de negocio: El mejor modelo obtenido mediante RandomizedSearchCV alcanzó en el conjunto de prueba un MAE ≈ 44.6 bicicletas/hora y un RMSE ≈ 64.5 bicicletas/hora, con un $R^2 \approx 0.869$.
  - El MAE indica que, en condiciones normales, la empresa debería esperar un error promedio cercano a 45 bicicletas por hora en sus predicciones de demanda. Dado que la demanda promedio en el dataset de Washington D.C.

ronda las ~190 bicicletas/hora, este error representa aproximadamente un 23–24% de desviación relativa.

- El R^2 cercano a 0.87 confirma que el modelo captura gran parte de la variabilidad de la demanda, lo que lo hace confiable para planificación operativa y presupuestal en escenarios regulares (asignación de turnos, redistribución de flota, estimación de ingresos). Sin embargo, el RMSE revela que en horas pico los errores pueden ser significativamente mayores, lo que implica riesgo de subestimación en momentos críticos. Para decisiones contractuales que requieran un nivel de servicio exacto, el modelo podría no ser suficiente por sí solo; sería recomendable complementarlo con datos adicionales (eventos especiales, clima en tiempo real) o modelos híbridos más precisos.

Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

1. Descripción del problema

En esta fase se realiza el cálculo manual de la propagación hacia adelante en una red neuronal artificial de tres capas (entrada, capa oculta y capa de salida), con el objetivo de clasificar una campaña publicitaria en una de las siguientes categorías:

- **C1:** Bajo Impacto
- **C2:** Impacto Medio
- **C3:** Impacto Alto

El vector de entrada está definido como:

$$X = [0.5, 0.3, 0.2]^T$$

2.1. Cálculo del vector lineal (Z(1))

$$W(1) = [[0.2, 0.5, -0.1], \qquad b(1) = [0.1, -0.2, 0.0]$$

$$[0.8, -0.3, 0.4],$$

$$[-0.5, 0.1, 0.2]]$$

$$X = [0.5, 0.3, 0.2]$$

$$Z1[1] = 0.2(0.5) + 0.5(0.3) + (-0.1)(0.2) + 0.1$$

$$= 0.10 + 0.15 - 0.02 + 0.10 = 0.33$$

$$Z1[2] = 0.8(0.5) + (-0.3)(0.3) + 0.4(0.2) + (-0.2)$$

$$= 0.40 - 0.09 + 0.08 - 0.20 = 0.19$$

$$Z1[3] = (-0.5)(0.5) + 0.1(0.3) + 0.2(0.2) + 0.0$$

$$= -0.25 + 0.03 + 0.04 + 0.00 = -0.18$$

$$Z(1) = [0.33, 0.19, -0.18]$$

Por lo tanto:

$$Z(1) = [0.33, 0.19, -0.18]$$

2.2 Función de activación (A(1))

Se aplica la función sigmoide a cada componente:

sigma(z) = 1 / (1 + e^(-z))

Resultados:

A1[1] = sigma(0.33) = 1/(1+e^-0.33) = 1/(1+0.7189) = 1/1.7189 = 0.5818

A1[2] = sigma(0.19) = 1/(1+e^-0.19) = 1/(1+0.8270) = 1/1.8270 = 0.5474

A1[3] = sigma(-0.18) = 1/(1+e^0.18) = 1/(1+1.1972) = 1/2.1972 = 0.4551

Por lo tanto:

$$A(1) = [0.5818, 0.5474, 0.4551]$$

3.1. Cálculo del vector lineal (Z(2)):

W(2) = [[0.5, -0.4, 0.1], b(2) = [-0.1, 0.2, 0.1]

[0.2, 0.8,-0.5],

[-0.1, 0.3, 0.7]]

A(1) = [0.5818, 0.5474, 0.4551]

Resultados:

Z2[1] = 0.5(0.5818)+(-0.4)(0.5474)+0.1(0.4551)+(-0.1)

= 0.2909 - 0.2190 + 0.0455 - 0.1000 = 0.0174

Z2[2] = 0.2(0.5818)+0.8(0.5474)+(-0.5)(0.4551)+0.2

= 0.1164 + 0.4379 - 0.2276 + 0.2000 = 0.5267

Z2[3] = (-0.1)(0.5818)+0.3(0.5474)+0.7(0.4551)+0.1

= -0.0582 + 0.1642 + 0.3186 + 0.1000 = 0.5246

Por lo tanto:

Z(2) = [0.0174, 0.5267, 0.5246]

4. Probabilidades de Clase (A(2)):

La función Softmax se utiliza para convertir los puntajes en probabilidades:

Softmax(zi) = e^zi / (e^z1 + e^z2 + e^z3)

Cálculo exponencial:

$$e^{0.0174} = 1.0176$$

$$e^{0.5267} = 1.6929$$

$$e^{0.5246} = 1.6893$$

$$\text{Suma} = 1.0176 + 1.6929 + 1.6893 = 4.3998$$

Probabilidades finales:

- P(C1 – Bajo Impacto): 0.2313 (23.1%)
- P(C2 – Impacto Medio): 0.3848 (38.5%)
- P(C3 – Impacto Alto): 0.3839 (38.4%)

4. Interpretación de resultados

La red neuronal clasifica la campaña publicitaria en la categoría C2 — Impacto Medio, con una probabilidad de 38.5%, seguida muy de cerca por la categoría C3 — Impacto Alto con 38.4%.

La diferencia entre ambas probabilidades es mínima (0.1 puntos porcentuales), lo que evidencia una alta incertidumbre en la decisión del modelo. Este comportamiento indica que la muestra se encuentra en una región limítrofe del espacio de clasificación.

Desde una perspectiva analítica, se puede concluir que la campaña presenta características híbridas entre impacto medio y alto, por lo que resulta más adecuado interpretarla como de impacto medio-alto, en lugar de una clasificación categórica estricta.

5. Preguntas conceptuales

1. ¿Por qué se utiliza Softmax en la capa de salida?

La función Softmax permite transformar los valores de salida en una distribución de probabilidad válida, garantizando que:

- Todas las probabilidades sean positivas.
- La suma total sea igual a 1.

A diferencia de la función sigmoide aplicada de forma independiente, Softmax introduce un mecanismo de competencia entre clases, donde el incremento en la probabilidad de una categoría implica la disminución de las demás. Esto resulta fundamental en problemas de clasificación multiclase, donde cada instancia debe pertenecer a una única categoría.

2. ¿Qué sucede si los sesgos b son cero?

Si los vectores de sesgo fueran nulos, la red neuronal quedaría restringida a combinaciones lineales puras de las entradas:

$$[Z = W X]$$

En este caso, cuando el vector de entrada es cero, la salida de todas las neuronas también sería cero, independientemente de los pesos. Esto limita significativamente la capacidad del modelo para representar patrones complejos.

El sesgo actúa como un parámetro de desplazamiento, permitiendo ajustar el umbral de activación de las neuronas. Su ausencia reduce la flexibilidad del modelo y dificulta el aprendizaje de relaciones no triviales en los datos.

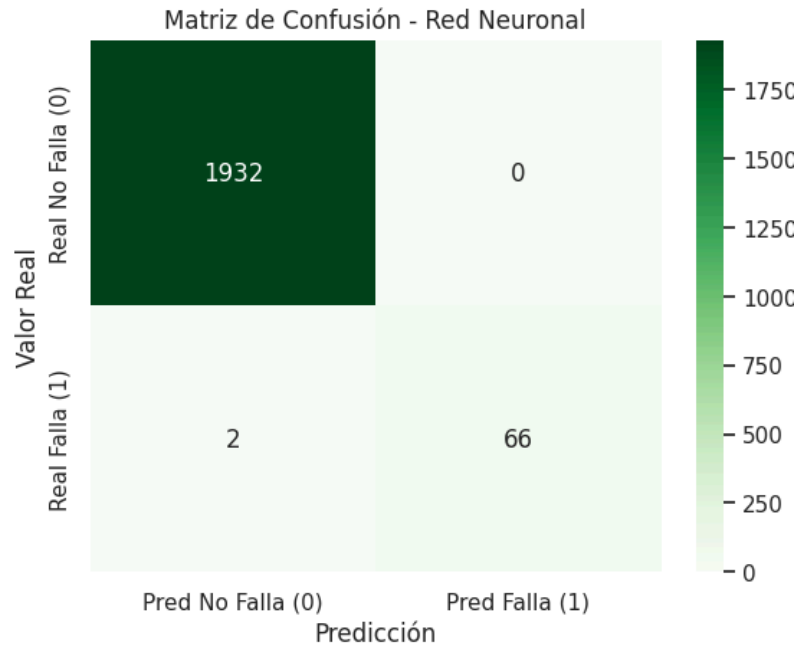
Fase 5.

Preguntas 5. A

- El class_weight busca compensar el desbalance de clases. En este ejercicio los fallos son muy pocos (menores al 5%), el modelo tendría a ignorar esos fallos y predecir un No Falla. Al darle un peso mayor a la clase minoritaria, el algoritmo penaliza más fuerte los errores en esa clase, obligando a la red a aprender patrones que permitan detectar fallos aunque sean raros. Para este trabajo se evidenció un recall de 97% (muy alto), lo cual es crítico en mantenimiento.

- Si el 95% de los registros son No Falla, un modelo que siempre prediga No Falla tendría un accuaracy del 95%. pero nunca detectará un fallo real. Lo importante no es la exactitud global, sino el recall de la clase de fallos, porque un fallo no detectado implica una parada de máquina y un costo muy alto.

```
--- Matriz de Confusión (Red Neuronal) ---
```



Preguntas reflexión y Negocio.

- $Precisión \ = \frac{TP}{TP+FP} = \frac{66}{66+0} = 100$

  $Recall \ = \frac{TP}{TP+FP} = \frac{66}{66+2} = 0.97$

  $Costo\ total\ error = (FN * 5000) + (FP * 200) = 10000\ USD$

  El modelo actual tiene un 97% de recall. Eso significa que casi todos los fallos se detectan, aunque en otros casos podría implicar falsas alarmas. Sin embargo, en mantenimiento predictivo siempre conviene priorizar el recall, porque el costo de un fallo no detectado es de 5,000 USD, mucho mayor que el de una falsa alarma que es 200 USD.

- ¿Considera que 30 épocas son suficientes para que el modelo converja?

Con 30 épocas, se observa que el modelo converge rápidamente. Ya desde la época 3-5 el recall supera el 95% y el accuracy alcanza valores cercanos al 99%. A partir de este punto, las métricas se mantienen estables hasta la época 30.

¿Qué sucede con el error de validación comparado con el de entrenamiento?

Comenzó en 0.57 en la primera época y disminuyó rápidamente hasta 0.0206 en la época 30. Además, el val_accuracy y el val_recall alcanzaron 1,0000 desde la época 2 y se mantuvieron estables hasta el final. Esto significa que el modelo generaliza correctamente, el error de validación se comporta de manera similar al de entrenamiento y se mantiene muy bajo. No hay señales de sobreajuste, ya que el error de validación nunca aumenta mientras el de entrenamiento baja.

Conclusión del trabajo.

La arquitectura óptima de una red neuronal no depende únicamente de su tamaño, sino del equilibrio entre complejidad y regularización. En este taller se evidenció que una red amplia con un nivel moderado de penalización (*alpha = 0.01*) y el uso de *early stopping* ofreció el mejor desempeño, evitando tanto el sobreajuste como el sobreajuste. En entornos industriales, este balance es esencial porque las decisiones derivadas de los modelos tienen impacto directo en costos y eficiencia operativa. Asimismo, las métricas de sensibilidad como el recall en mantenimiento predictivo y el RMSE en predicción de demanda resultan más relevantes que la exactitud global, ya que permiten identificar los errores más críticos: fallos poco frecuentes o subestimaciones en horas pico. En conclusión, la combinación de una arquitectura adecuada y métricas de sensibilidad robustas asegura que los modelos no solo sean precisos, sino también útiles para la toma de decisiones en escenarios reales.

Referencias.

[1] Moreno Quintero, E., & Muñoz, G. C. (2021). Técnicas de regularización en el marco del aprendizaje de máquina: Regresiones Ridge y Lasso [Monografía de trabajo de grado, Universidad Distrital Francisco José de Caldas]. https://repository.udistrital.edu.co/bitstreams/ebfc4323-d49c-41c2-be17-e4375323bdcb/download

[2] Piña Monarrez, M. R., Rodríguez Medina, M. A., & Aguirre Solís, J. J. (2007). Regresión Ridge y la distribución central t. Ciencia Ergo Sum, 14(2), 191–196. https://dialnet.unirioja.es/descarga/articulo/5116534.pdf

[3] Jiménez Márquez, J. L. (2019). Definición de un Framework para el Análisis Predictivo de Datos no Estructurados [Tesis doctoral, Universidad Carlos III de Madrid]. https://e-archivo.uc3m.es/bitstream/10016/29210/1/tesis_jose-luis_jimenez_marquez_2019.pdf

[4] Edgar, T. F., Himmelblau, D. M., & Lasdon, L. S. (2001). Optimization of chemical processes (2nd ed.). McGraw-Hill. https://gymkhana.iitb.ac.in/~scp/scp/ocw/chemical/cl%20603%20optimization/Optimization%20of%20Chemical%20Processes%20-%20Edgar%20Himmelblau%20and%20Lasdon%202nd%20ed.pdf

[5] González, P., & Pérez, P. (s. f.). Tema 3: Entrenamiento de redes neuronales profundas. Universidad de Oviedo. https://pglez82.github.io/DeepLearningWeb/slides/tema3.pdf

[6] Caicedo Bravo, E. F., & López Sotelo, J. A. (2009). Una aproximación práctica a las redes neuronales artificiales. Programa Editorial Universidad del Valle.

https://bibliotecadigital.univalle.edu.co/bitstream/handle/10893/10330/Una-aproximacion-practica-a-las-redes.pdf?sequence=6

[7] Santillana Quesada, S. (2022). Introducción a las redes neuronales para el tratamiento de imágenes [Trabajo de fin de grado, Universidad de Granada]. https://digibug.ugr.es/bitstream/handle/10481/76426/libro%281%29.pdf?isAllowed=y&sequence=1

[8] IBM Corp. (s. f.). IBM SPSS Statistics 28: Redes neuronales. https://www.ibm.com/docs/SSLVMB_28.0.0/pdf/es/IBM_SPSS_Neural_Network.pdf

[9] Tapia, S. (2024). Mantenimiento predictivo basado en machine learning: Una revisión sistemática de la literatura y perspectivas en la industria 4.0. Avances en Ciencia e Ingeniería. https://acijournal.cl/index.php/ojs/article/view/3

[10] Martínez Alonso, G. (2023). Machine learning aplicado al mantenimiento predictivo de un sistema hidráulico [Trabajo de fin de grado, Universidad Europea de Madrid]. https://hdl.handle.net/20.500.12880/7753

[11] Fanaee-T, H. (2013). Bike Sharing [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5W894

[12] Scikit-learn developers. (s. f.). Ridge. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html

[13] Scikit-learn developers. (s. f.). MLPRegressor. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html

[14] Scikit-learn developers. (s. f.). RandomizedSearchCV. scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[15] Scikit-learn developers. (s. f.). Metrics and scoring: Quantifying the quality of predictions. scikit-learn. https://scikit-learn.org/stable/modules/model_evaluation.html